

TP02 : Les structures de contrôle en Python

Adrien ESCOURROU

November 19, 2025

INTRODUCTION

Ce TP02 est consacré aux structures de contrôle en Python. L'objectif de ce travail est de comprendre et d'appliquer les mécanismes de décision et de répétition qui permettent à un programme d'exécuter différemment des blocs de code en fonction de conditions et de boucles. Nous explorerons les instructions if, elif et else, les boucles for et while, ainsi que les notions associées telles que les opérateurs de comparaison, les opérateurs logiques et les conventions de lisibilité du code. À l'issue du TP, vous serez capables d'écrire des programmes plus flexibles et plus efficaces, capables de s'adapter à différentes entrées et scénarios.

Rappels essentiels

Pour faire une suite d'instructions il est préférable d'utiliser la syntaxe for :

```
1  for e in sequence:
2      instruction 1
3      ...
4      instruction n
5  # retour a la ligne sans indentation
```

Pour faire une suite d'instructions mais avec une condition d'arrêt, il faut utiliser while :

```
1  while condition :
2      instruction 1
3      ...
4      instruction n
5  # retour a la ligne sans indentation
```

Pour ajouter des conditions dans mon itération, il faut rajouter des if/else :

```
1  if condition 1:
2      Traitement 1
3  elif condition 2:
4      Traitement 2
5  ...
6  elif condition 3:
7      Traitement 3
8  else:
9      Traitement n
10 # retour a la ligne sans indentation
```

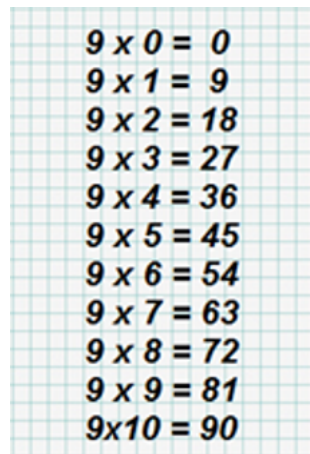
Depuis Python 3.10, il est recommandé d'utiliser match/case :

```
1      match valeur:
2          case pattern_1:
3              expression_1
4          case pattern_2:
5              expression_2
6          case pattern_3:
7              expression_3
8          case _:
9              expression_par_defaut
```

*Tous vos résultats devront être publiés sur votre répertoire GIT partagé avec votre professeur en TP01. Ici vous nommerez un répertoire **TP02** et annoterez chaque code avec l'exercice associé. Par exemple, pour l'exercice 3 vous publierez **Exo3.py** dans **TP02**.*

Exercice 1

Écrire un script Python pour afficher la table de multiplication par 9



9	x	0	=	0
9	x	1	=	9
9	x	2	=	18
9	x	3	=	27
9	x	4	=	36
9	x	5	=	45
9	x	6	=	54
9	x	7	=	63
9	x	8	=	72
9	x	9	=	81
9	x	10	=	90

Figure 1: Table de mutiplication

Exercice 2

On plante un arbre qui mesure 30 cm et qui pousse de 14 cm par an. Écrire un programme en Python à l'aide d'une boucle for pour savoir combien il mesure au bout de 8 ans.



Figure 2: Arbre

Exercice 3

Inès veut construire une pyramide à base carrée comme sur la photo ci-dessous. La pyramide sur la photo a 7 étages.

1. Inès veut construire une pyramide à 100 étages. Combien de billes doit-elle avoir ? Écrire un programme en Python pour répondre au problème.
2. Modifiez le programme pour que l'utilisateur puisse choisir le nombre d'étages.



Figure 3: Pyramide

Exercice 4

En mathématiques, la suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Elle commence généralement par les termes 0 et 1 (parfois 1 et 1) et ses premiers termes sont 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc. Écrivez un programme paramétrique qui fournit la série de Fibonacci pour obtenir les valeurs de n termes.

Le quiz magique

Le programme doit obligatoirement utiliser :

- *Au moins une boucle while*
- *Au moins une boucle for*
- *Au moins une structure if / elif / else*

Description générale :

1. Au démarrage, le programme demande à l'utilisateur (le professeur) :
Combien de questions il veut poser à l'élève (par exemple 5, 10, ...) ?
2. Le programme lance ensuite un quiz pour l'élève :
 - Pour chaque question, il affiche une opération simple (addition, soustraction ou multiplication) entre deux petits nombres.
 - L'élève saisit sa réponse au clavier.
 - Le programme vérifie si la réponse est correcte et affiche un message adapté :
 - « *Correct !* » si la réponse est bonne
 - « *Faux, la bonne réponse était ...* » sinon
3. À la fin du quiz, le programme affiche :
 - Le nombre total de bonnes réponses
 - Le pourcentage de réussite de l'élève
 - Un message d'appréciation selon le score, par exemple :
 - Plus de 80 % : « *Bravo !* »
 - Entre 50 % et 80 % : « *Pas mal, continue à t'entraîner.* »
 - Moins de 50 % : « *Il faut réviser encore.* »
4. Le programme propose ensuite un menu (en boucle) :

1	1 - Refaire un quiz
2	2 - Voir le meilleur score obtenu jusqu'a present
3	3 - Quitter

 - Si l'utilisateur choisit 1 : on relance un nouveau quiz (on peut redemander le nombre de questions).

- Si l'utilisateur choisit 2 : on affiche le meilleur pourcentage de réussite obtenu depuis le lancement du programme.
- Si l'utilisateur choisit 3 : le programme s'arrête.
- Pour tout autre choix : afficher un message d'erreur et redemander une option.

Exemple d'exécution (simplifié)

```
1 Combien de questions pour ce quiz ? 5
2
3 Question 1 : 3 + 4 = ?
4 Votre reponse : 7
5 Correct !
6
7 ...
8
9 Vous avez 4 bonnes reponses sur 5.
10 Votre pourcentage de reussite est : 80.0 %
11 Bravo !
12
13 Menu :
14 1 - Refaire un quiz
15 2 - Voir le meilleur score
16 3 - Quitter
17 Votre choix : 2
18
19 Meilleur score jusqu'a present : 80.0 %
20
21 Menu :
22 1 - Refaire un quiz
23 2 - Voir le meilleur score
24 3 - Quitter
25 Votre choix : 3
26
27 Au revoir !
```

Conclusion

Bravo pour vos premiers programmes Python. Merci de tout publier sur GIT.